

Práctica 2: Criptografía Simétrica y Seguridad Perfecta

Javier López Cano

Mario García Pascual

November 25, 2020

1 Seguridad Perfecta

Para implementar un programa (seg-perf.c) que pruebe que un cifrado por desplazamiento tiene seguridad perfecta si sus claves son equiprobables, y no tiene seguridad perfecta si las claves no son equiprobables, nos hemos basado en la definición de seguridad perfecta. Un criptosistema tiene seguridad perfecta si el texto plano es independiente del texto cifrado, de este modo, obtener el texto cifrado no nos da ninguna información acerca del contenido del texto plano. Matemáticamente esto significa:

$$P_x(x|y) = P_x(x) \forall x, y \in \mathbb{Z}_m$$

Donde x es el caracter del texto plano, y el caracter correspondiente del texto cifrado, y $\mathbb{Z}_m$ el alfabeto de m caracteres.

Partiendo de este principio, ciframos un texto con un cifrado por desplazamiento, nosotros hemos empleado el texto de El Quijote proporcionado en el archivo "quijote". Para cifrar este texto, simplemente obtenemos claves con una función *key_gen* que genera números pseudoaleatorios mediante la función "mpz_urandomm" de "gmp". En caso de querer mantener la equiprobabilidad de claves, pasamos a esta función de "gmp" el número 26 para que nos genere números aleatorios entre 0 y 25 de forma equiprobable; si por el contrario queremos que las claves no sean equiprobables, pasamos el número 37, y aplicamos módulo 26 al número obtenido. Cualquier número no múltiplo de 26 (tamaño del alfabeto) escogido para pasar a la función de "gmp" modificaría las probabilidades de las claves al aplicar el módulo 26, pero hemos escogido concretamente el 37 pues es el que mayor desviación produce en la probabilidad de las claves generadas.

```
1 int key_gen(gmp_randstate_t state){
2     mpz_t key, max;
3     int final_key, module;
4     if(mode == NEP) module = 37;
5     else module = 26;
6     mpz_init(key);
7     mpz_init(max);
8     mpz_set_ui(max, module);
9     mpz_urandomm(key, state, max);
10    final_key = mpz_get_ui(key);
11    mpz_clear(max);
12    mpz_clear(key);
13    return final_key;
14 }
```

Tras cifrar el texto, procedemos a calcular todas las probabilidades $P_x(x)$ y $P_x(x|y)$. Para ello almacenamos las $P_x(x)$ en un array con 26 entradas, una por cada posible caracter del alfabeto, y vamos sumando las apariciones de cada caracter en el texto plano y almacenando el total en la posición correspondiente del array, al finalizar el texto, dividimos estos valores entre la longitud total del texto. Para las $P_x(x|y)$ realizamos un proceso similar, almacenando las probabilidades en una tabla de tamaño 26x26 donde la fila indica el caracter del texto plano, y la columna el del texto cifrado; de este modo vamos recorriendo ambos textos simultáneamente y nos fijamos en los pares (x, y) que aparecen en la misma posición del texto plano y cifrado respectivamente, contamos las repeticiones de estos pares y las almacenamos en la posición $[x][y]$ de la tabla, finalmente dividimos todos los valores entre la $P_y(y)$ correspondiente que hemos ido calculando del mismo modo que $P_x(x)$ pero sobre el texto cifrado, de este modo obtenemos las probabilidades deseadas en la tabla.

En este proceso calculamos también para cada x , la media y desviación de la diferencia $|P_x(x) - P_x(x|y)|$, pues cuanto menor sean estos valores, más nos aproximamos a $P_x(x) = P_x(x|y)$.

```

1 void calc_probs(){
2     int i, j, c, c_enc;
3     for(i = 0; i < ALPH_SIZE; i++){
4         for(j = 0; j < ALPH_SIZE; j++){
5             probs_cond[i][j] = 0;
6         }
7     while ((c = fgetc(in)) != EOF && IS_IN_ALPH(c)){
8         probs[c - 'A']++;
9         total++;
10        if ((c_enc = fgetc(out)) != EOF && IS_IN_ALPH(c_enc)){
11            probs_cond[c_enc - 'A'][c - 'A']++;
12            freqs_enc[c_enc - 'A']++;
13        }else{
14            printf("Texto cifrado incorrecto.\n");
15        }
16    }
17    for(i = 0; i < ALPH_SIZE; i++){
18        probs[i] /= total;
19    }
20    for(i = 0; i < ALPH_SIZE; i++){
21        for(j = 0; j < ALPH_SIZE; j++){
22            probs_cond[i][j] /= freqs_enc[i];
23        }
24    }
25    for(i = 0; i < ALPH_SIZE; i++){
26        for(j = 0; media[i] = 0; j < ALPH_SIZE; j++){
27            media[i] += fabs(probs_cond[j][i] - probs[i]);
28        }
29        media[i] /= ALPH_SIZE;
30    }
31    for(i = 0; i < ALPH_SIZE; i++){
32        for(j = 0; desv[i] = 0; j < ALPH_SIZE; j++){
33            desv[i] += pow(fabs(probs_cond[j][i] - probs[i]) - media[i]), 2);
34        }
35        desv[i] /= ALPH_SIZE;
36        desv[i] = sqrt(desv[i]);
37    }
38 }

```

Procedemos entonces a imprimir estas probabilidades, medias y desviaciones, y, a la vez, calculamos la media total de la diferencia $|P_x(x) - P_x(x|y)|$ para todo x, y , pues estos valores son los que nos darán una visión más global de lo cerca que se halla el criptosistema de la seguridad perfecta.

```

1 void print_probs(){
2     int i, j;
3     double med = 0;
4     for(i = 0; i < ALPH_SIZE; i++){
5         printf("-----");
6         printf("\nP(%c) = %lf\n", i + 'A', probs[i]);
7         for(j = 0; j < ALPH_SIZE; j++){
8             if(j % 2 == 0) printf("\n");
9             printf("P(%c|%c) = %lf\t", i + 'A', j + 'A', probs_cond[j][i]);
10        }
11        printf("\n\nError medio: %lf\t", media[i]);
12        printf("Desviacion del error: %lf\n", desv[i]);
13        med += media[i];
14    }
15    printf("-----");
16    printf("\n\nError medio total: %lf\n", med/ALPH_SIZE);
17 }

```

1.1 Análisis de resultados

Al aplicar este programa sobre el texto de El Quijote antes mencionado, obtenemos el siguiente resultado cuando empleamos claves equiprobables:

```

P(Y|M) = 0.013472      P(Y|N) = 0.013578
P(Y|O) = 0.014980      P(Y|P) = 0.015396
P(Y|Q) = 0.015369      P(Y|R) = 0.014457
P(Y|S) = 0.014293      P(Y|T) = 0.014981
P(Y|U) = 0.014849      P(Y|V) = 0.015267
P(Y|W) = 0.016791      P(Y|X) = 0.015373
P(Y|Y) = 0.015106      P(Y|Z) = 0.015045

Error medio: 0.000566   Desviación del error: 0.000455
-----
P(Z) = 0.003938

P(Z|A) = 0.003316      P(Z|B) = 0.003827
P(Z|C) = 0.003749      P(Z|D) = 0.004190
P(Z|E) = 0.003799      P(Z|F) = 0.004249
P(Z|G) = 0.004379      P(Z|H) = 0.004048
P(Z|I) = 0.003782      P(Z|J) = 0.004197
P(Z|K) = 0.002887      P(Z|L) = 0.004226
P(Z|M) = 0.004006      P(Z|N) = 0.003621
P(Z|O) = 0.003843      P(Z|P) = 0.004149
P(Z|Q) = 0.003698      P(Z|R) = 0.004343
P(Z|S) = 0.004385      P(Z|T) = 0.004154
P(Z|U) = 0.004256      P(Z|V) = 0.003558
P(Z|W) = 0.003570      P(Z|X) = 0.004680
P(Z|Y) = 0.003479      P(Z|Z) = 0.004012

Error medio: 0.000318   Desviación del error: 0.000219
-----
Error medio total: 0.000720
eps@labvirteps:~/Escritorio/Cripto/p2$ █

```

Figure 1: Salida del programa seg-perf para El Quijote -P

La salida que produce el programa es muy amplia, y es difícil verla completa en la pantalla al mismo tiempo, sin embargo, el fragmento que más nos interesa es la línea final, que nos indica la media de la diferencia $|P_x(x) - P_x(x|y)|$, mencionada anteriormente, observamos que esta es muy próxima a cero, lo que nos indica que estamos muy próximos a la seguridad perfecta. También podemos observar esto en los medios de esta diferencia para cada uno de los x , pues vemos que estas son también muy bajas, o incluso mirando a las $P_x(x|y)$ específicas y comparándolas con el $P_x(x)$ correspondiente, ya que se puede apreciar que en todos los casos la diferencia es mínima.

Comparamos ahora con la salida que obtenemos para el mismo texto con claves no equiprobables:

```

P(Y|O) = 0.010638      P(Y|P) = 0.011125
P(Y|Q) = 0.011560      P(Y|R) = 0.009903
P(Y|S) = 0.009508      P(Y|T) = 0.010278
P(Y|U) = 0.009737      P(Y|V) = 0.009214
P(Y|W) = 0.010652      P(Y|X) = 0.011165
P(Y|Y) = 0.021591      P(Y|Z) = 0.023712

Error medio: 0.005549   Desviación del error: 0.001313
-----
P(Z) = 0.003938

P(Z|A) = 0.005184      P(Z|B) = 0.005633
P(Z|C) = 0.005207      P(Z|D) = 0.005185
P(Z|E) = 0.005803      P(Z|F) = 0.005915
P(Z|G) = 0.006006      P(Z|H) = 0.005919
P(Z|I) = 0.004870      P(Z|J) = 0.005061
P(Z|K) = 0.002802      P(Z|L) = 0.002828
P(Z|M) = 0.003017      P(Z|N) = 0.002871
P(Z|O) = 0.002326      P(Z|P) = 0.002756
P(Z|Q) = 0.003254      P(Z|R) = 0.002428
P(Z|S) = 0.002571      P(Z|T) = 0.002688
P(Z|U) = 0.002738      P(Z|V) = 0.002809
P(Z|W) = 0.002899      P(Z|X) = 0.002689
P(Z|Y) = 0.002355      P(Z|Z) = 0.007252

Error medio: 0.001414   Desviación del error: 0.000514
-----
Error medio total: 0.013040

```

Figure 2: Salida del programa seg-perf para El Quijote -I

Es fácilmente apreciable que la media de esta diferencia es mucho mayor en este caso, siendo superior hasta en un orden 2. Esto nos muestra como nos hemos alejado de la seguridad perfecta simplemente variando la equiprobabilidad de las claves, que es justo lo que queríamos demostrar. Observando las probabilidades $P(x|y)$ específicas y las medias para cada x , se puede llegar a la misma conclusión pues observamos que las medias para cada x también son significativamente superiores al caso anterior, y las diferencias entre los $P_x(x|y)$ y los $P_x(x)$ también son mayores, y tienen valores mucho más variados, lo que se muestra también por los valores de dispersión más elevados. Alguna excepción se observa en los caracteres menos frecuentes como la "W" o la "K", ya que tienen tan pocas apariciones en el texto plano que todas sus probabilidades, son muy próximas a cero independientemente de la y , por lo que, aun con claves no equiprobables, la diferencia entre la probabilidad condicionada y la probabilidad del caracter en el texto plano es mínima.

Para ver como afecta el tamaño del texto a estas probabilidades, y por tanto a la seguridad, empleamos el mismo programa en un texto de menor tamaño, en nuestro caso hemos empleado El Lazarillo de Tormes, proporcionado en el fichero "lz.txt", en el caso con claves equiprobables obtenemos:

```

P(Y|G) = 0.020063      P(Y|H) = 0.016986
P(Y|I) = 0.020847      P(Y|J) = 0.022131
P(Y|K) = 0.019298      P(Y|L) = 0.019565
P(Y|M) = 0.025373      P(Y|N) = 0.017118
P(Y|O) = 0.021789      P(Y|P) = 0.019108
P(Y|Q) = 0.017110      P(Y|R) = 0.018027
P(Y|S) = 0.024390      P(Y|T) = 0.017805
P(Y|U) = 0.017577      P(Y|V) = 0.019669
P(Y|W) = 0.018424      P(Y|X) = 0.020748
P(Y|Y) = 0.026130      P(Y|Z) = 0.014963

Error medio: 0.002069   Desviación del error: 0.001784
-----
P(Z) = 0.004255

P(Z|A) = 0.004653      P(Z|B) = 0.005381
P(Z|C) = 0.007683      P(Z|D) = 0.003108
P(Z|E) = 0.004695      P(Z|F) = 0.004459
P(Z|G) = 0.004702      P(Z|H) = 0.002831
P(Z|I) = 0.003849      P(Z|J) = 0.003794
P(Z|K) = 0.004429      P(Z|L) = 0.004418
P(Z|M) = 0.003172      P(Z|N) = 0.004046
P(Z|O) = 0.004878      P(Z|P) = 0.004459
P(Z|Q) = 0.004753      P(Z|R) = 0.002846
P(Z|S) = 0.003484      P(Z|T) = 0.005180
P(Z|U) = 0.002197      P(Z|V) = 0.004683
P(Z|W) = 0.005400      P(Z|X) = 0.003772
P(Z|Y) = 0.002459      P(Z|Z) = 0.005299

Error medio: 0.000865   Desviación del error: 0.000722
-----
Error medio total: 0.002317
eps@labvirtips:~/Escritorio/Cripto/p2$

```

Figure 3: Salida del programa seg-perf para El Lazarillo de Tormes -P

Podemos ver que al reducirse el tamaño del texto, aumentan notablemente las variaciones en las probabilidades, esto se debe a que, en general, las probabilidades tienden a estabilizarse cuanto mayor es el número de datos que se tienen para observar. A pesar de esto, la media continua siendo notablemente baja, si tomásemos un texto aún más pequeño, sin duda ese "Error medio" aumentaría aún más, y también aumentarían las desviaciones.

Comprobamos ahora el caso con claves no equiprobables, del mismo modo que con las claves equiprobables, al ser un texto más pequeño, las probabilidades serán menos fiables al tener menor cantidad de datos, por lo que es probable que se reduzcan las diferencias entre este caso y el caso con claves equiprobables respecto a las que había en El Quijote. El resultado que obtenemos es:

```

P(Y|M) = 0.015590      P(Y|N) = 0.011149
P(Y|O) = 0.013018      P(Y|P) = 0.017247
P(Y|Q) = 0.013258      P(Y|R) = 0.015453
P(Y|S) = 0.010182      P(Y|T) = 0.014484
P(Y|U) = 0.010966      P(Y|V) = 0.017788
P(Y|W) = 0.014523      P(Y|X) = 0.011519
P(Y|Y) = 0.032415      P(Y|Z) = 0.033004

Error medio: 0.006958   Desviación del error: 0.003351
-----
P(Z) = 0.004255

P(Z|A) = 0.007866      P(Z|B) = 0.004811
P(Z|C) = 0.006917      P(Z|D) = 0.003330
P(Z|E) = 0.008926      P(Z|F) = 0.005215
P(Z|G) = 0.006367      P(Z|H) = 0.004997
P(Z|I) = 0.003905      P(Z|J) = 0.004884
P(Z|K) = 0.002449      P(Z|L) = 0.002300
P(Z|M) = 0.002545      P(Z|N) = 0.003716
P(Z|O) = 0.003028      P(Z|P) = 0.002929
P(Z|Q) = 0.001989      P(Z|R) = 0.003154
P(Z|S) = 0.006171      P(Z|T) = 0.002465
P(Z|U) = 0.003557      P(Z|V) = 0.002965
P(Z|W) = 0.004236      P(Z|X) = 0.002491
P(Z|Y) = 0.003566      P(Z|Z) = 0.006125

Error medio: 0.001507   Desviación del error: 0.001010
-----

Error medio total: 0.012899
eps@labvirteps:~/Escritorio/Cripto/p2$

```

Figure 4: Salida del programa seg-perf para El Lazarillo de Tormes -I

Como esperábamos, la diferencia entre los casos con claves equiprobables y con claves no equiprobables es, en este texto, tan solo de orden 1, mientras que en El Quijote, significativamente más largo, era de orden 2. Del mismo modo, al tomar textos más pequeños, esta diferencia se iría reduciendo aún más hasta ser prácticamente inapreciable debido al reducido número de datos disponibles para calcular las probabilidades, que acrecenta las diferencias entre probabilidades que, en un conjunto de datos mayor, serían mucho más próximas.

1.2 Uso del programa

Hemos seguido el formato especificado en el enunciado, que es el siguiente:

```
seg-perf {-P|-I} [-i file_in] [-o file_out]
```

donde los parámetros son:

- el modo: `-P` o `-I`, para claves equiprobables o no equiprobables respectivamente.
- el fichero de entrada: `-i` seguido del nombre del fichero. Si no se introduce se usa la entrada estándar.
- el fichero de salida: `-o` seguido del nombre del fichero. Si no se introduce se usa la salida estándar.

Un ejemplo de uso cifrando *El Quijote* con claves equiprobables:

```
1 ./seg-perf -P -i quijote -o quijote_enc
```

Que nos proporcionaría la salida mostrada en la primera imagen del análisis de resultados.

2 Implementación del DES

2.1 Programación del DES

En este problema se nos pide hacer una implementación del DES con el modo de operación CBC. A continuación comentaremos algunos asuntos que menciona el enunciado.

Padding. En nuestro programa se van leyendo bloques de 8 bytes hasta leer uno incompleto. Entonces, se rellenan los bytes que no han sido leídos con bytes aleatorios y se procede a la última iteración del algoritmo.

Generación de claves. Cuando ejecutamos el programa en modo cifrado no hay que introducir ni la clave ni el vector de inicialización, pues el programa los generará por nosotros y los mostrará por pantalla. El vector de inicialización se crea con 8 bytes aleatorios. La clave también son 8 bytes aleatorios pero estableciendo los bits de paridad impar. Se entiende *bit de paridad impar* de un conjunto de bits como un bit que se establece a 1 cuando el número de bits a 1 es par, y a 0 en caso contrario. De modo que el conjunto de bits 1111111 tiene bit de paridad impar a 0, mientras que 0000000 lo tiene a 1.

ECB para mensajes largos y estructurados. Hemos implementado el programa `desECB`, que ejecuta mismo algoritmo pero usando el modo de operación ECB. El formato de uso es el mismo salvo porque por razones evidentes no acepta vector de inicialización. Luego, lo hemos utilizado para cifrar el contenido de una imagen del logo de Moodle. (Cifrar el contenido de una imagen y poder seguir visualizándola requiere unas sutilezas técnicas que no tiene sentido desarrollar aquí.)



Figure 5: Logo de Moodle

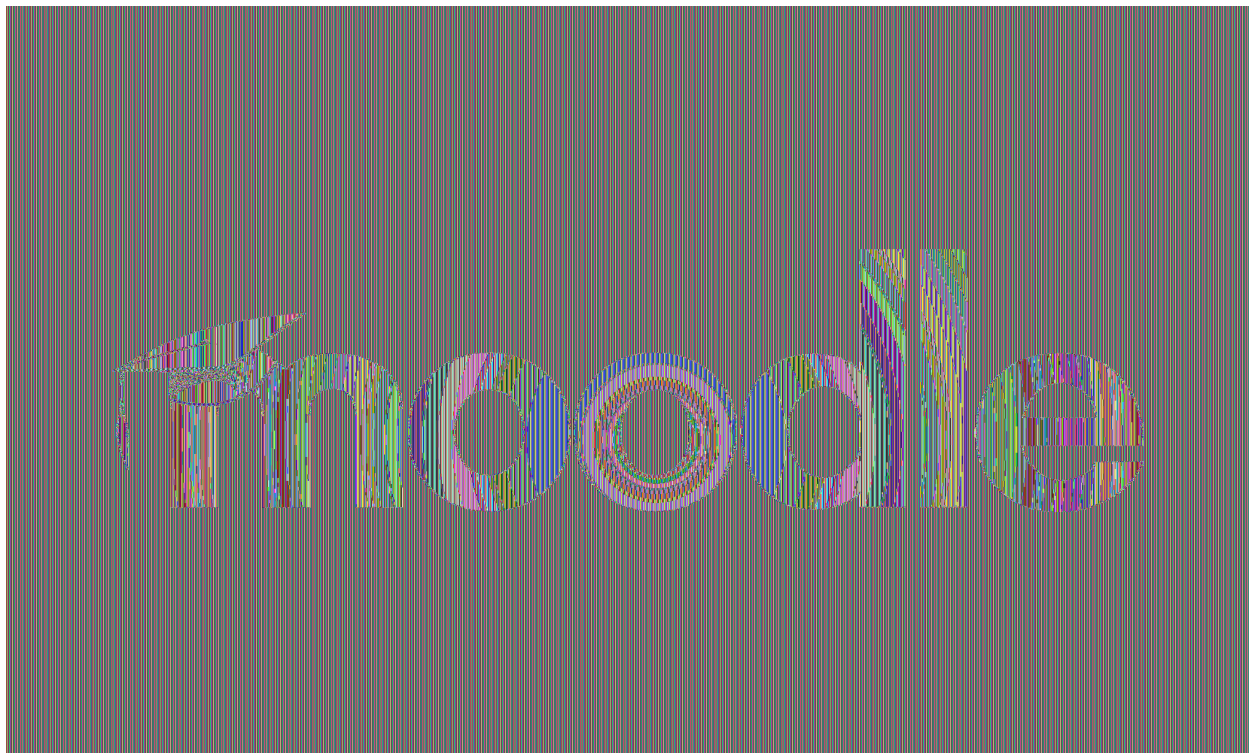


Figure 6: Logo de Moodle cifrado con el modo de operación ECB



Figure 7: Logo de Moodle cifrado con el modo de operación CBC

Como se puede observar, el resultado de cifrar con CBC no se puede distinguir a simple vista del ruido blanco, mientras que el obtenido con ECB nos permite conocer el contenido de la imagen que se cifró. Esto se debe

a que en ECB bloques iguales se cifran igual, mientras que en CBC el cifrado del bloque t depende de el historial de bloques hasta el tiempo $t - 1$, además del vector de inicialización. En el caso de las imágenes, es posible que, como en el caso de los logos, haya grandes espacios del mismo color y que por tanto se cifrarán igual, permitiendo reconocer a simple vista contenido de la imagen plana. Esto ocurre de igual manera en otros tipos de mensajes largos y estructurados.

The DES Algorithm Illustrated. Para implementar el algoritmo nos ha sido útil la web de *The DES Algorithm Illustrated*. Por ello hemos implementado el programa `desIllustrated`, que no recibe argumentos y simplemente se dedica a imprimir por pantalla el contenido de todas las variables intermedias que aparecen en la web. Esto nos permite comprobar que nuestra implementación hace coincidir hasta el último bit del último byte, y que por tanto está implementado correctamente, al menos en lo que se refiere al modo de operación ECB. De todos modos, más adelante comprobaremos la implementación comparando nuestro programa con OpenSSL.

2.1.1 Uso del programa

Hemos mantenido el formato del enunciado, añadiendo una opción de verbosidad.

```
desCBC {-C|-D} [-k clave] [-iv vector inic.] [-i file_in] [-o file_out] [-v]
```

donde los parámetros son:

- el modo: `-C` para cifrado y `-D` para descifrado.
- la clave: `-k` seguido de la clave de 8 bytes en hexadecimal.
- el vector de inicialización: `-iv` seguido del vector de inicialización de 8 bytes en hexadecimal.
- el fichero de entrada: `-i` seguido del nombre del fichero. Si no se introduce se usa la entrada estándar.
- el fichero de salida: `-o` seguido del nombre del fichero. Si no se introduce se usa la salida estándar.
- la verbosidad: si se añade `-v` el programa imprimirá información adicional de la ejecución de algoritmo.

A continuación mostramos un ejemplo de uso normal y las salidas que se obtienen:

```
./desCBC -C -i q -o qcif
Modo C/D: 0
Clave: 0000000000000000
IV: 0000000000000000
Entrada: q
Salida: qcif
-----
GENERATED KEY: 67832c91d304dacd
GENERATED IV: 919dde337f38ff7d

./desCBC -D -k 67832c91d304dacd -iv 919dde337f38ff7d -i qcif -o qcifdes
Modo C/D: 1
Clave: 67832c91d304dacd
IV: 919dde337f38ff7d
Entrada: qcif
Salida: qcifdes

head -n 2 qcifdes
DON QUIJOTE DE LA MANCHA
Miguel de Cervantes Saavedra
```

- `q`: El Quijote
- `qcif`: El Quijote cifrado
- `qcifdes`: EL Quijote cifrado y descifrado. Con un poco de suerte coincidirá con El Quijote, excepto, quizás, por unos caracteres aleatorios de *padding* al final del texto.

En cuanto a la salida del programa: primero mostramos los argumentos con los que se ha ejecutado el programa. Después, en el caso de ejecutar en modo cifrado, se imprimirá la clave y el vector de inicialización; en caso de ejecutar en modo descifrado, no se imprime información adicional. No comentamos la salida si se activa el modo verboso porque lo consideramos autoexplicativo y, además, de escasa utilidad.

2.1.2 Corrección de la implementación

Para comprobar la corrección de nuestro trabajo hemos decidido comparar la salida con OpenSSL. Dado un fichero, si ciframos y desciframos su contenido con las claves adecuadas recuperamos el texto original, más algunos bytes aleatorios al final si se ha requerido de padding. Esto podría ocurrir incluso habiendo cometido errores en la implementación el algoritmo, por ejemplo, implementado un algoritmo distinto a DES con modo de operación CBC pero que forma un criptosistema. En esta sección probamos que el algoritmo implementado es DES CBC, y no otro, comparando la salida con el cifrado adecuado de OpenSSL y viendo que coincide en todos los bytes, excepto, quizás, en los 8 últimos. Después comprobamos el chequeo de los bits de paridad impares.

2.1.3 Comparación con OpenSSL

Ya que estamos, comprobamos también que el programa que implementa ECB funciona correctamente.

```
./desECB -C -i q -o qcif
Modo C/D: 0
Clave: 0000000000000000
Entrada: q
Salida: qcif
-----
GENERATED KEY: b95e1f135d0eae6b
```

El modo de OpenSSL para DES con modo de operación ECB es `des-ecb`:

```
openssl enc -des-ecb -in q -out qcif.enc -nosalt -K b95e1f135d0eae6b
```

```
hexdump -C qcif -n48
00000000 51 59 b2 29 3a 0a 33 26 00 85 be 29 1a ee f6 45 |QY.):.3&...)...E|
00000010 49 33 e7 6c 50 99 b0 24 57 f6 69 ea 82 c4 ec 5b |I3.lP..$W.i....[|
00000020 74 70 db 60 55 4e c8 d0 66 35 5e d3 c1 47 0a 49 |tp.'UN..f5^..G.I|
00000030
```

```
hexdump -C qcif.enc -n48
00000000 51 59 b2 29 3a 0a 33 26 00 85 be 29 1a ee f6 45 |QY.):.3&...)...E|
00000010 49 33 e7 6c 50 99 b0 24 57 f6 69 ea 82 c4 ec 5b |I3.lP..$W.i....[|
00000020 74 70 db 60 55 4e c8 d0 66 35 5e d3 c1 47 0a 49 |tp.'UN..f5^..G.I|
00000030
```

Comprobamos con hexdump que los primeros 48 bytes de los ficheros coinciden.

```
./desCBC -C -i q -o qcif
Modo C/D: 0
Clave: 0000000000000000
IV: 0000000000000000
Entrada: q
Salida: qcif
-----
GENERATED KEY: ad1c1a1c31579798
GENERATED IV: 0748bf35b85c964e
```

El modo de OpenSSL para DES con modo de operación CBC es `des-cbc`:

```
openssl enc -des-cbc -in q -out qcif.enc -nosalt -K ad1c1a1c31579798 -iv 0748bf35b85c964e
```

```
hexdump -C qcif -n48
00000000 c9 fe 6a 8c 98 5a a3 bd 6c 17 c1 50 63 69 53 ff |..j..Z..l..PciS.|
00000010 8a 83 3a 84 60 e6 11 d1 67 a0 fb 1f 75 2b 70 5f |...'....g....u+p_|
00000020 ef 8e 38 04 b0 f1 fb 0f 7c 05 b6 fe a5 93 a4 26 |..8.....|.....&|
00000030
```

```
hexdump -C qcif.enc -n48
00000000 c9 fe 6a 8c 98 5a a3 bd 6c 17 c1 50 63 69 53 ff |..j..Z..l..PciS.|
00000010 8a 83 3a 84 60 e6 11 d1 67 a0 fb 1f 75 2b 70 5f |...'....g....u+p_|
00000020 ef 8e 38 04 b0 f1 fb 0f 7c 05 b6 fe a5 93 a4 26 |..8.....|.....&|
00000030
```

Comprobamos con hexdump que los primeros 48 bytes de los ficheros coinciden.

2.1.4 Comprobación bits de paridad

Mostramos como cambiando un bit cualquiera de una clave correcta el programa nos avisa de que la clave no pasa el chequeo de los bits de paridad y finaliza la ejecución.

```
./desCBC -C -i q -o qcif
Modo C/D: 0
Clave: 0000000000000000
IV: 0000000000000000
Entrada: q
Salida: qcif
-----
GENERATED KEY:      cb62b6bf7083ef5b
GENERATED IV:       9d878aac0a43c008

./desCBC -D -k db62b6bf7083ef5b -iv 9d878aac0a43c008 -i qcif -o qcifdes
Modo C/D: 1
Clave: db62b6bf7083ef5b
IV: 9d878aac0a43c008
Entrada: qcif
Salida: qcifdes
Error: bits de paridad incorrectos
```

Nótese como hemos cambiado la clave de cb62b6bf7083ef5b a db62b6bf7083ef5b, modificando el primer byte en un solo bit.

2.2 Programación del Triple DES

Pasamos directamente a el uso del programa y a comprobar que está implementado correctamente.

2.2.1 Uso del programa

En esta ocasión sí que hemos modificado ligeramente el formato que se nos da en el enunciado. Hemos cambiado el nombre del programa a `tdeaCBC` y para facilitar la implementación hemos decidido separar el parámetro de la clave en 3 distintos. El formato final queda de la siguiente forma:

```
tdeaCBC {-C|-D} [-k1 clave1] [-k2 clave2] [-k3 clave3] [-iv vector inic.] [-i file_in] [-o file_out] [-v]
```

donde los parámetros son:

- el modo: `-C` para cifrado y `-D` para descifrado.
- las claves: `-k1`, `-k2` y `-k3`, cada uno seguido de la clave de 8 bytes en hexadecimal.
- el vector de inicialización: `-iv` seguido del vector de inicialización de 8 bytes en hexadecimal.
- el fichero de entrada: `-i` seguido del nombre del fichero. Si no se introduce se usa la entrada estándar.
- el fichero de salida: `-o` seguido del nombre del fichero. Si no se introduce se usa la salida estándar.
- la verbosidad: si se añade `-v` el programa imprimirá información adicional de la ejecución de algoritmo.

El uso del programa es exactamente el mismo que el programa anterior. Incluimos ejemplos de su uso, esta vez sin comentarios añadidos.

```
./tdeaCBC -C -i q -o qcif
Modo C/D: 0
Clave 1: 0000000000000000
Clave 2: 0000000000000000
Clave 3: 0000000000000000
IV: 0000000000000000
Entrada: q
Salida: qcif
-----
GENERATED KEYS:
      c1322904b362010e
      3e92c485e6516d46
      5e97b3c8ef370d6d

FOR SLL:      c1322904b362010e3e92c485e6516d465e97b3c8ef370d6d

GENERATED IV:  99f3518b7c5f5748
```

```
./tdeaCBC -D -k1 c1322904b362010e -k2 3e92c485e6516d46 -k3 5e97b3c8ef370d6d -iv 99f3518b7c5f5748 -i qcif
-o qcifdes
Modo C/D: 1
Clave 1: c1322904b362010e
Clave 2: 3e92c485e6516d46
Clave 3: 5e97b3c8ef370d6d
IV: 99f3518b7c5f5748
Entrada: qcif
Salida: qcifdes

head -n 2 qcifdes
DON QUIJOTE DE LA MANCHA
Miguel de Cervantes Saavedra
```

2.2.2 Comparación con OpenSSL

Lo mismo que para el apartado de DES simple, vamos a comparar la salida con cifrado con OpenSSL para asegurarnos que el algoritmo que estamos implementando es Triple DES con tres claves y modo de ejecución CBC, y no otro.

```
./desCBC -C -i q -o qcif
Modo C/D: 0
Clave: 0000000000000000
IV: 0000000000000000
Entrada: q
Salida: qcif
-----
GENERATED KEY: ad1c1a1c31579798
GENERATED IV: 0748bf35b85c964e
```

El modo de OpenSSL para Triple DES con tres claves y modo de operación CBC es `des-ede3-cbc`:

```
openssl enc -des-ede3-cbc -in q -out qcif.enc -nosalt -K 382c1a40670bcd04a410a76defcd73199de3cdd99110bcd3
-iv 3d826d37761a11fa
```

```
hexdump -C -n 48 qcif
00000000 7f d3 cb 19 87 16 3d 39 9b 6b 69 af 6a 9b 32 a1 |.....=9.ki.j.2.|
00000010 f4 da b3 d9 63 8f bc 0d fb d8 e6 82 e9 53 c6 f5 |....c.....S..|
00000020 75 d0 50 00 c3 05 b4 fe a1 f8 94 df 12 2c 94 d1 |u.P.....,..|
00000030

hexdump -C -n 48 qcif.enc
00000000 7f d3 cb 19 87 16 3d 39 9b 6b 69 af 6a 9b 32 a1 |.....=9.ki.j.2.|
00000010 f4 da b3 d9 63 8f bc 0d fb d8 e6 82 e9 53 c6 f5 |....c.....S..|
00000020 75 d0 50 00 c3 05 b4 fe a1 f8 94 df 12 2c 94 d1 |u.P.....,..|
00000030
```

Comprobamos con hexdump que los primeros 48 bytes de los ficheros coinciden.

2.2.3 Comprobación bits de paridad

Para el tema de los bits de paridad, está implementado del mismo modo que antes y funciona igual.

3 Principios de diseño del DES

3.1 Estudio de la no linealidad de las S-boxes del DES

En este problema reutilizamos las funciones del problema anterior para comprobar, de modo exhaustivo, la no linealidad de cada una de las S-Boxes. En concreto, la función que nos interesa es

```
uint64_t s_box(uint64_t input, int s)
```

que recibe un input de 6 bits y aplica la $(s + 1)$ -ésima S-Box.

El programa sencillamente itera sobre todos los posibles inputs de cada S-Box y comprueba si se preserva o no la operación $\oplus$. A saber, para cada $x, y \in \{b_1b_2b_3b_4b_5b_6 : b_i = 0, 1\}$ y para cada S_i se comprueba si

$$S_i(x \oplus y) = S_i(x) \oplus S_i(y),$$

obteniendo un índice de no-linealidad para cada S_i y un índice de no-linealidad total, que se imprimen por pantalla al final de la ejecución.

3.1.1 Uso del programa

El formato del programa es el siguiente:

```
desSBNL [-v]
```

desSBNL significa DES S-Boxes Non-Linearity, y sólo recibe un argumento opcional `-v` para activar el modo verboso. Si este modo está activado se imprimen las entradas que son lineales. No incluimos un ejemplo en la memoria.

El resultado de ejecutar el programa es el siguiente:

```
./desSBNL
----- Resultados
----- Tabla con indice de no-linealidad de cada S-Box:
S-BOX 1 S-BOX 2 S-BOX 3 S-BOX 4 S-BOX 5 S-BOX 6 S-BOX 7 S-BOX 8
0.9404 0.9399 0.9380 0.9424 0.9414 0.9390 0.9375 0.9370
----- Indice de no-linealidad total: 0.9395
```

Observamos un índice de no-linealidad del 93,95%. La S-Box 4 alcanza el máximo de no-linealidad con un 94,24%, mientras que el mínimo lo logra la S-Box 8 en 93,7%.

3.2 Estudio del Efecto de Avalancha

Se nos pide comprobar el efecto avalancha del cifrado DES. Se conoce como efecto avalancha a la propiedad deseable en cifrados consistente en que cambios mínimos en bloque de texto plano que se cifra producen cambios mayores en el bloque cifrado. Y lo mismo para la clave: cambios mínimos en la clave producen, sobre bloques iguales, cifrados muy diferentes.

Para comprobar esto, generamos un bloque y una clave aleatoriamente, modificamos un bit de aleatoriamente de uno de ellos, y monitorizamos como se va desencadenando la avalancha viendo el contenido de $L_i R_i$ para $i = 0, 1, \dots, 16$. Si $L_i R_i$ son el contenido de las variables para el cifrado de original y $L'_i R'_i$ para el cifrado alterado, nos fijamos en el número de bits que tienen en común en cada ronda.

Por ejemplo, la diferencia entre $L_0 R_0$ y $L'_0 R'_0$ será de exactamente un bit en caso que estemos comprobando el efecto avalancha sobre bloques, y 0 cuando lo comprobemos sobre la clave. Con un poco de suerte, la diferencia subirá a 32 bits rápidamente y se oscilará cerca de este valor, pues 32 bits es la diferencia media que se puede esperar entre dos bloques aleatorios.

3.2.1 Uso del programa

El formato del programa es el siguiente:

```
desAvalancha [-v]
```

El único de parámetro del programa es opcional y activa el modo verboso, que imprime información adicional durante la ejecución del algoritmo. La información adicional es el valor de $L_i R_i$ y $L'_i R'_i$ en bits, para poder comprobar que la diferencia en bits que indica el programa en cada ronda es correcta.

Ahora mostramos el resultado de una ejecución:

```
./desAvalancha
----- Diferencia bits/Ronda cambiando 1 bit aleatorio del bloque
ROUND 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
BITDIFF 1 1 8 23 33 29 28 31 32 40 44 36 31 34 33 33 35
----- Diferencia bits/Ronda cambiando 1 bit aleatorio de la clave
ROUND 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
BITDIFF 0 3 14 25 31 33 27 24 29 29 32 38 34 30 30 28 26
```

La primera tabla indica el número de bits de diferencia entre LR y $L'R'$ habiendo cambiado aleatoriamente un bit del bloque a cifrar. La segunda tabla es lo mismo pero cambiando aleatoriamente un bit de la clave. Podemos observar que a partir de la ronda 4 se alcanza el valor deseado de 32 bits, que es la diferencia media entre dos bloques aleatorios.

Nota. Aproximadamente un octavo de las ejecuciones la segunda tabla aparecerá con una diferencia de bits 0 constante. Esto se debe a que al cambiar un bit aleatoriamente de la clave, hay una probabilidad de $1/8$ de modificar uno de los bits de paridad, que será consecuentemente ignorado al realizar el algoritmo, y el resultado del cifrado será el mismo que para la otra clave.

4 Principios de diseño del AES

En esta sección hemos implementado 2 programas que comprueban y emplean los principios de diseño del AES, el primero de estos "AES_lineal_check.c" comprueba la no-linealidad de las S-Boxes del AES; el segundo "AES_table_calc.c" calcula las S-Boxes directa e inversa del algoritmo.

Concretamente para la implementación del segundo programa mencionado, hemos creado una librería auxiliar "gf8.c" que implementa la división y el producto en $GF(2^8)$, así como el algoritmo de euclides normal y extendido para este campo. Esta librería implementa también 2 funciones, "descomp_gf" que transforman un entero (de hasta 8 bits) en un array de enteros que contiene su representación binaria, y "recomp_gf" que realiza el proceso inverso, transformando el array de enteros de vuelta a número entero.

4.1 Estudio de la no-linealidad de las S-Boxes del AES

Para probar la no linealidad de las S-Boxes, debemos probar que el acceso a esta no cumple la linealidad respecto al XOR, que es la operación equivalente a la suma en $GF(2^8)$. Como notación, para este apartado, llamaremos $f(x)$ al valor de la tabla correspondiente a la entrada x , donde x es un entero positivo entre 0 y 255 (ambos incluidos) cuyos 4 bits más significativos denotan la fila de la tabla a la que hay que acceder, y los 4 menos significativos indican la columna.

De este modo, en el programa "AES_lineal_check", hemos implementado una función "search_table_AES", que dado un entero de entrada (x), y una tabla donde buscar (DIRECT = 0 indica la tabla directa e INVERSE = 1 la inversa), devuelve el valor correspondiente de la tabla para esa entrada, para ello toma como índice de la fila el valor $x/16$ (división entera), y como índice de la columna $x \bmod 16$, y con estos parámetros obtiene el valor correspondiente de la tabla directa o inversa del AES proporcionada en el fichero "AES_tables.c" que se nos proporcionaba en el moodle de la asignatura.

```
1 long search_table_AES(int input, int tabla){
2     int fila = input / 16;
3     int columna = input % 16;
4     char *ptr;
5
6     if(tabla == DIRECT){
7         return strtol(DIRECT_SBOX[fila][columna], &ptr, 16);
8     }else if(tabla == INVERSE){
9         return strtol(INVERSE_SBOX[fila][columna], &ptr, 16);
10    }else{
11        printf("Tabla no valida.");
12        return ERR;
13    }
14 }
```

Tras esto implementamos la función "check_table" que, dada una tabla y 2 parámetros de entrada (enteros de hasta 8 bits) comprueba si esa tabla cumple la linealidad para esos parámetros, es decir, comprueba si:

$$f(x1) \oplus f(x2) = f(x1 \oplus x2)$$

Para ello simplemente obtiene los valores $f(x1)$ y $f(x2)$ de la tabla correspondiente realiza el XOR entre ellos con el operando bit a bit de C " $\oplus$ ", opera con este mismo operador $x1 \oplus x2$, y obtiene de la tabla $f(x1 \oplus x2)$, tras lo cual compara ambos valores para determinar si son o no iguales.

```
1 int check_table(int input1, int input2, int tabla){
2     int op1, op2, res_sum, res_prod, sum, prod;
3
4     printf("#####\n");
5
6     if(tabla == DIRECT){
7         printf("Comprobando no-linealidad tabla directa del AES con sumandos %02X y %02X:\n", input1, input2);
8     }else if(tabla == INVERSE){
9         printf("Comprobando no-linealidad tabla inversa del AES con sumandos %02X y %02X:\n", input1, input2);
10    }
11
12    printf("#####\n");
13
14    op1 = search_table_AES(input1, tabla);
15    if(op1 == ERR) return ERR;
16    op2 = search_table_AES(input2, tabla);
```

```

17  if(op2 == ERR) return ERR;
18  printf("f(%02X) = %02X\n", input1, op1);
19  printf("f(%02X) = %02X\n", input2, op2);
20  printf("\nf(%02X) XOR f(%02X) = %02X\n", input1, input2, op1^op2);
21
22  sum = input1^input2;
23  res_sum = search_table_AES(sum, tabla);
24  if(res_sum == ERR) return ERR;
25  printf("f(%02X XOR %02X) = f(%02X) = %02X\n", input1, input2, sum, res_sum);
26  if(res_sum == (op1 ^ op2)){
27      printf("\nCumple linealidad en XOR. f(x) XOR f(y) = f(x XOR y)\n\n");
28      return 100;
29  }else{
30      printf("\nNo cumple linealidad en XOR. f(x) XOR f(y) != f(x XOR y)\n\n");
31  }
32  return OK;
33 }

```

La rutina principal del programa simplemente genera todos los pares posibles de enteros de 8 bits y llama a la función "check_table" con todos los pares en ambas tablas. Aunque bastaría con que la propiedad $f(x_1) \oplus f(x_2) = f(x_1 \oplus x_2)$ no se cumpliera en tan solo un valor para probar que las tablas no cumplen la linealidad respecto a XOR; como deben ser altamente no lineales hemos decidido probar todos los posibles pares y ver para qué porcentaje de estos sí cumple la linealidad. Por tanto, el "main" además de comprobar las tablas con todos los pares, calcula el porcentaje de estos para los que las tablas sí cumplen la linealidad respecto al XOR.

Al ejecutar el programa se obtiene una salida muy amplia que se puede dividir fácilmente en 3 segmentos. Un primer segmento en el que para cada par de enteros (x_1, x_2) y para cada S-Box del AES se imprimen los resultados de la comprobación de la linealidad:

```

#####
Comprobando no-linealidad tabla inversa del AES con sumandos FF y CA:
#####
f(FF) = 7D
f(CA) = 10

f(FF) XOR f(CA) = 6D
f(FF XOR CA) = f(35) = D9

No cumple linealidad en XOR. f(x) XOR f(y) != f(x XOR y)

```

Tras esto se imprimen todos los pares de datos para los que se cumple la linealidad en la tabla directa, y luego los de la inversa:

La tabla directa cumple linealidad en XOR con los siguientes pares de datos:

(03, 54)	(03, 57)	(06, E0)	(06, E6)	(07, 11)	(07, 16)
(08, 17)	(08, 1F)	(0A, C1)	(0A, CB)	(0B, 15)	(0B, 1E)
(0C, F4)	(0C, F8)	(0D, 93)	(0D, 9E)	(0E, A2)	(0E, AC)
(10, C0)	(10, D0)	(11, 07)	(11, 16)	(12, 6B)	(12, 79)
(15, 0B)	(15, 1E)	(16, 07)	(16, 11)	(17, 08)	(17, 1F)
(19, 25)	(19, 3C)	(1B, A8)	(1B, B3)	(1D, A3)	(1D, BE)
(1E, 0B)	(1E, 15)	(1F, 08)	(1F, 17)	(22, 4A)	(22, 68)
(25, 19)	(25, 3C)	(26, C3)	(26, E5)	(29, 92)	(29, BB)
(2B, 45)	(2B, 6E)	(32, 44)	(32, 76)	(33, 41)	(33, 72)
(34, 95)	(34, A1)	(35, D7)	(35, E2)	(36, 5B)	(36, 6D)
(3B, 4B)	(3B, 70)	(3C, 19)	(3C, 25)	(3D, 58)	(3D, 65)
(3E, 99)	(3E, A7)	(41, 33)	(41, 72)	(42, AF)	(42, ED)
(44, 32)	(44, 76)	(45, 2B)	(45, 6E)	(47, BA)	(47, FD)
(4A, 22)	(4A, 68)	(4B, 3B)	(4B, 70)	(4F, BF)	(4F, F0)
(53, A0)	(53, F3)	(54, 03)	(54, 57)	(57, 03)	(57, 54)
(58, 3D)	(58, 65)	(5B, 36)	(5B, 6D)	(61, A4)	(61, C5)
(62, B7)	(62, D5)	(63, BC)	(63, DF)	(64, 8D)	(64, E9)
(65, 3D)	(65, 58)	(67, 83)	(67, E4)	(68, 22)	(68, 4A)
(69, 90)	(69, F9)	(6B, 12)	(6B, 79)	(6C, A6)	(6C, CA)
(6D, 36)	(6D, 5B)	(6E, 2B)	(6E, 45)	(70, 3B)	(70, 4B)
(72, 33)	(72, 41)	(73, 9B)	(73, E8)	(76, 32)	(76, 44)
(77, 82)	(77, F5)	(79, 12)	(79, 6B)	(7B, 81)	(7B, FA)
(7E, AD)	(7E, D3)	(7F, 98)	(7F, E7)	(81, 7B)	(81, FA)
(82, 77)	(82, F5)	(83, 67)	(83, E4)	(8D, 64)	(8D, E9)
(90, 69)	(90, F9)	(92, 29)	(92, BB)	(93, 0D)	(93, 9E)
(95, 34)	(95, A1)	(98, 7F)	(98, E7)	(99, 3E)	(99, A7)
(9B, 73)	(9B, E8)	(9E, 0D)	(9E, 93)	(A0, 53)	(A0, F3)
(A1, 34)	(A1, 95)	(A2, 0E)	(A2, AC)	(A3, 1D)	(A3, BE)
(A4, 61)	(A4, C5)	(A6, 6C)	(A6, CA)	(A7, 3E)	(A7, 99)
(A8, 1B)	(A8, B3)	(AC, 0E)	(AC, A2)	(AD, 7E)	(AD, D3)
(AF, 42)	(AF, ED)	(B3, 1B)	(B3, A8)	(B7, 62)	(B7, D5)
(BA, 47)	(BA, FD)	(BB, 29)	(BB, 92)	(BC, 63)	(BC, DF)
(BE, 1D)	(BE, A3)	(BF, 4F)	(BF, F0)	(C0, 10)	(C0, D0)
(C1, 0A)	(C1, CB)	(C3, 26)	(C3, E5)	(C5, 61)	(C5, A4)
(CA, 6C)	(CA, A6)	(CB, 0A)	(CB, C1)	(D0, 10)	(D0, C0)
(D3, 7E)	(D3, AD)	(D5, 62)	(D5, B7)	(D7, 35)	(D7, E2)
(DF, 63)	(DF, BC)	(E0, 06)	(E0, E6)	(E2, 35)	(E2, D7)
(E4, 67)	(E4, 83)	(E5, 26)	(E5, C3)	(E6, 06)	(E6, E0)
(E7, 7F)	(E7, 98)	(E8, 73)	(E8, 9B)	(E9, 64)	(E9, 8D)
(ED, 42)	(ED, AF)	(F0, 4F)	(F0, BF)	(F3, 53)	(F3, A0)
(F4, 0C)	(F4, F8)	(F5, 77)	(F5, 82)	(F8, 0C)	(F8, F4)
(F9, 69)	(F9, 90)	(FA, 7B)	(FA, 81)	(FD, 47)	(FD, BA)

Por último aparecen los porcentajes de los pares para los que se cumple la linealidad en cada una de las tablas.

Esto supone que:

```
La tabla directa es lineal en XOR para 252 pares de 65536 posibles, un 0.384521%
La tabla inversa es lineal en XOR para 252 pares de 65536 posibles, un 0.384521%
javi@javi-TP301UA:~/Documentos/Cripto/p2$
```

Observando los resultados, se ve que el porcentaje de pares para los que se cumple la linealidad respecto al XOR es extremadamente pequeño, por tanto podemos afirmar que las S-Boxes de AES son altamente no-lineales.

4.1.1 Uso del programa

Este es un programa extremadamente sencillo de ejecutar al no tener ningún parámetro de entrada, basta con ejecutar en la terminal:

```
1 ./AES_lineal_check
```

Y se producirá la salida descrita.

4.2 Cálculo de las S-Boxes del AES

Para calcular las S-Boxes hemos implementado el producto y la división en $GF(2^8)$ en la librería "gf8.c".

4.2.1 Librería gf8.c

La división se ha implementado en la función "gf_div" a la que se le pasan 2 enteros cuya representación en binario son los coeficientes de los 2 polinomios GF a dividir, y devuelve una estructura con el cociente y el resto de la división en el mismo formato (enteros que representan los coeficientes de los polinomios). En primer lugar comprobamos si a (dividendo) es menor que b (divisor) en cuyo caso el resto es a y el cociente 0. Despues, mientras a tenga una longitud mayor que b quitamos dígitos binarios de la derecha dividiendo el entero entre 2) y contamos el número de dígitos quitados en un contador, y cuando a y b tienen la misma longitud multiplicamos el cociente por 2 y le sumamos 1 y aplicamos un XOR entre a y b, ese resultado es el nuevo resto. Bajamos uno de los caracteres que hemos quitado, para ello multiplicamos el resto por 2 y sumamos el valor (0 o 1) del dígito a bajar (lo obtenemos mediante operaciones con módulos y potencias de 2). Tras esto comparamos este nuevo resto con b y mientras el resto tenga menos dígitos que b seguimos bajando los dígitos que hemos quitado y añadiendo 0s al cociente (multiplicandolo por 2), cuando el resto sea de longitud binaria mayor o igual a la de b realizamos de nuevo el XOR y repetimos el proceso. Este proceso terminará cuando, al ir a bajar uno de los dígitos binarios de a, el contador donde acumulamos los dígitos que quedan por bajar "count_size" esté a 0, que indica que no quedan dígitos por bajar; en ese momento devolvemos el resto y el cociente en la estructura.

```
1 struct div gf_div(int a, int b){
2     struct div result;
3     int a_aux, b_aux, count_size=0, i, cmp;
4     result.cociente = 0;
5     result.resto = a;
6     if(a < b){
7         return result;
8     }
9
10    while(1){
11        for(i=0; i < 32; i++){
12            cmp = (int)pow(2, i);
13            if(cmp > result.resto){
14                break;
15            }
16        }
17        if(cmp <= b){
18            result.cociente *= 2;
19            if(count_size == 0){
20                return result;
21            }
22            a_aux = (a % (int)pow(2, (double)count_size));
23            a_aux -= (a_aux % (int)pow(2, (double)count_size - 1));
24            if(a_aux == 0){
25                result.resto = result.resto * 2;
26            }else{
27                result.resto = result.resto * 2 + 1;
28            }
29            count_size--;
30            continue;
31        }
32        result.cociente = result.cociente * 2 + 1;
33        a_aux = result.resto;
34        for(i=0; i < 32; i++){
35            cmp = (int)pow(2, i);
36            if(cmp > b){
37                break;
38            }
39        }
40        while(cmp <= a_aux){
41            a_aux = floor((double)a_aux/2);
42            count_size++;
43        }
44
45        result.resto = a_aux ^ b;
46        if(count_size == 0){
47            return result;
48        }
49        a_aux = (a % (int)pow(2, (double)count_size));
50        a_aux -= (a_aux % (int)pow(2, (double)count_size - 1));
```

```

51     if(a_aux == 0){
52         result.resto = result.resto * 2;
53     }else{
54         result.resto = result.resto * 2 + 1;
55     }
56     count_size--;
57 }
58 }

```

Para implementar el producto primero implementamos la función `xtime`, que recibe un entero que representa un polinomio $GF(2^8)$ y lo transforma de forma que el entero resultante represente ese polinomio multiplicado por x . Para ello, si el polinomio de entrada tiene coeficiente 0 para x^7 simplemente hace un desplazamiento a la izquierda (multiplicando por 2), si por el contrario, el coeficiente de x^7 es 1, además de desplazar a la izquierda realizamos un XOR con el valor 0x11B, que en decimal es 283.

```

1  int xtime(int a){
2      int aux, res;
3      aux = (a % (int)pow(2, 8));
4      aux -= (aux % (int)pow(2, (double)7));
5      if(aux == 0){
6          res = a * 2;
7          return res;
8      }else{
9          res = (a * 2) ^ 283;
10         return res;
11     }
12 }

```

Tras esto implementamos el producto en `gf_mul`, para ello descomponemos el entero a (primer factor) en un array que contiene su representación binaria empleando la función auxiliar `descomp_gf`. Tras esto inicializamos el resultado final `res` a 0 y recorremos el array calculando $a = xtime(a)$ cada vez que avanzamos una posición, y si el elemento de la posición i del array, que representa 2^i está a 1, realizamos un XOR entre el resultado del `xtime` y `res`, tras recorrer las 8 posiciones del array, devolvemos el entero resultante en `res`.

```

1  int gf_mul(int a, int b){
2      int i, *desc, res = 0, ret = 0;;
3      desc = descomp_gf(b);
4      if(desc[0] == 1){
5          ret = a;
6      }
7      res = a;
8      for(i=1; i<8; i++){
9          res = xtime(res);
10         if(desc[i] == 1){
11             ret = ret ^ res;
12         }
13     }
14     free(desc);
15     return ret;
16 }

```

Para implementar estas operaciones, y para permitir mayor flexibilidad posteriormente en el programa, se implementan 2 funciones muy sencillas `descomp_gf` y `recomp_gf` que transforman un entero (de hasta 8 bits) en su representación binaria almacenada en un array y viceversa.

Por último esta librería implementa las funciones para realizar el algoritmo de euclides y el algoritmo de euclides extendido, que se implementan del mismo modo que en la práctica anterior (aunque sin GMP), pero sustituyendo la división por `gf_div`, el producto por `gf_mul`, y la suma y la resta por XOR bit a bit. También se implementa por comodidad la función `inv_gf8` que dado un entero, llama a euclides extendido para devolver el inverso en $GF(2^8)$.

4.2.2 Programa principal SBOX_AES

Este programa se implementa en el archivo `AES_tables_calc.c` y consta de 3 funciones principales, y la rutina `main`, además de algunas funciones auxiliares.

En primer lugar la función `print_matrix` simplemente imprime una matriz (en este caso las S-Boxes calculadas) representada en un array doble de enteros, la imprime en un fichero, con los valores de los enteros en su representación hexadecimal.

La función "affine_transform" que dado un entero, calcula su transformación afín empleando la matriz constante X y el vector constante C del AES (enteros estáticos) si se ha escogido hallar la S-Box directa, y empleando $Y = X^{-1}$ y D (también almacenados en macros) si se ha escogido la S-Box inversa. Para ello simplemente se recorre el array con la representación binaria del entero (obtenido con "desc_gf") y la matriz correspondiente y se realiza el producto, tras lo cual se realiza un XOR del resultado con el vector C o D correspondiente, y devuelve de vuelta el entero resultante (obtenido a partir del array con "recomp_gf").

```

1 int affine_transf(int x){
2     int i, j, ret, *desc, *res;
3     desc = descomp_gf(x);
4     res = (int*)malloc(8 * sizeof(int));
5     for(i=0; i<8; i++){
6         res[i] = 0;
7         for(j=0; j<8; j++){
8             if(type == DIRECT){
9                 res[i] = res[i] ^ (X[i][j] * desc[j]);
10            }else{
11                res[i] = res[i] ^ (Y[i][j] * desc[j]);
12            }
13        }
14    }
15    if(type == DIRECT){
16        res[i] = res[i] ^ C[i];
17    }else{
18        res[i] = res[i] ^ D[i];
19    }
20 }
21 free(desc);
22 return recomp_gf(res);
23 }

```

La última función relevante es "compare_matrix" que dada una matriz (en este caso la S-Box calculada), la compara con la S-Box correspondiente almacenada en el fichero "AES_tables.c" proporcionado en el moodle de la asignatura, y comprueba si ambas son o no iguales. Compara con DIRECT_SBOX si se ha escogido calcular la S-Box directa, y con INVERSE_SBOX en caso contrario.

```

1 int compare_matrix(int **m){
2     int i, j;
3     char *ptr;
4     for(i=0; i<16; i++){
5         for(j=0; j<16; j++){
6             if(type == DIRECT){
7                 if(m[i][j] == (int)strtol(DIRECT_SBOX[i][j], &ptr, 16)){
8                     continue;
9                 }else{
10                    return 1;
11                }
12            }else if(type == INVERSE){
13                if(m[i][j] == (int)strtol(INVERSE_SBOX[i][j], &ptr, 16)){
14                    continue;
15                }else{
16                    return 1;
17                }
18            }else{
19                printf("type no valido.\n");
20                return -1;
21            }
22        }
23    }
24    return 0;
25 }

```

Basándose en estas funciones, el programa main lee los parámetros de entrada y almacena el tipo "DIRECT" o "INVERSE" en la variable "type", inicializa la matriz de 16 filas y 16 columnas que contendrá la S-Box a calcular, y tras eso comienza a calcularla. Para ello genera todos los números del 0 al 255 (ambos incluidos) en el contador "i", y los divide en sus dos dígitos hexadecimales, el primero es $line = i/16$ que representa la fila de la tabla a rellenar, y el segundo es $col = 1 \bmod 16$ que indica la columna de la tabla a rellenar. Una vez tenemos estos índices para indexar la tabla calculamos el valor a introducir. Si $type = DIRECT$ el valor a introducir se calcula hallando el inverso de "i" en $GF(2^8)$ con la función "inv_gf8" de "gf8.c", y obteniendo la transformación afín del resultado para la S-Box directa (empleando X y C) con la función "affine_transform". Si por el contrario $type = INVERSE$, invertimos el orden, de modo que el valor a introducir en la tabla se obtendrá realizando la transformación afín (con Y y D) del entero i, y obteniendo el inverso del resultado en

$GF(2^8)$. Introducimos el valor calculado en la posición [line][col] de la tabla.

Tras calcular toda la tabla, la imprime en el fichero de salida (o en stdout si este no se ha proporcionado) mediante la función "print_matrix", y comprueba que el resultado es correcto con la función "compare_matrix".

4.2.3 Análisis de resultados

Al ejecutar el programa para la tabla directa obtenemos el siguiente resultado:

```
javi@javi-TP301UA:~/Documentos/Cripto/p2$ ./SB0X_AES -C
Tipo de tabla C/D: C Directa
Salida: stdout
{ 63 7c 77 7b f2 6b 6f c5 30 01 67 2b fe d7 ab 76 }
{ ca 82 c9 7d fa 59 47 f0 ad d4 a2 af 9c a4 72 c0 }
{ b7 fd 93 26 36 3f f7 cc 34 a5 e5 f1 71 d8 31 15 }
{ 04 c7 23 c3 18 96 05 9a 07 12 80 e2 eb 27 b2 75 }
{ 09 83 2c 1a 1b 6e 5a a0 52 3b d6 b3 29 e3 2f 84 }
{ 53 d1 00 ed 20 fc b1 5b 6a cb be 39 4a 4c 58 cf }
{ d0 ef aa fb 43 4d 33 85 45 f9 02 7f 50 3c 9f a8 }
{ 51 a3 40 8f 92 9d 38 f5 bc b6 da 21 10 ff f3 d2 }
{ cd 0c 13 ec 5f 97 44 17 c4 a7 7e 3d 64 5d 19 73 }
{ 60 81 4f dc 22 2a 90 88 46 ee b8 14 de 5e 0b db }
{ e0 32 3a 0a 49 06 24 5c c2 d3 ac 62 91 95 e4 79 }
{ e7 c8 37 6d 8d d5 4e a9 6c 56 f4 ea 65 7a ae 08 }
{ ba 78 25 2e 1c a6 b4 c6 e8 dd 74 1f 4b bd 8b 8a }
{ 70 3e b5 66 48 03 f6 0e 61 35 57 b9 86 c1 1d 9e }
{ e1 f8 98 11 69 d9 8e 94 9b 1e 87 e9 ce 55 28 df }
{ 8c a1 89 0d bf e6 42 68 41 99 2d 0f b0 54 bb 16 }

Comparando S-Box con la del fichero AES_tables.c

Las S-Boxes coinciden.

javi@javi-TP301UA:~/Documentos/Cripto/p2$
```

Y con la tabla inversa:

```
javi@javi-TP301UA:~/Documentos/Cripto/p2$ ./SBOX_AES -D
Tipo de tabla C/D: D Inversa
Salida: stdout
{ 52 09 6a d5 30 36 a5 38 bf 40 a3 9e 81 f3 d7 fb }
{ 7c e3 39 82 9b 2f ff 87 34 8e 43 44 c4 de e9 cb }
{ 54 7b 94 32 a6 c2 23 3d ee 4c 95 0b 42 fa c3 4e }
{ 08 2e a1 66 28 d9 24 b2 76 5b a2 49 6d 8b d1 25 }
{ 72 f8 f6 64 86 68 98 16 d4 a4 5c cc 5d 65 b6 92 }
{ 6c 70 48 50 fd ed b9 da 5e 15 46 57 a7 8d 9d 84 }
{ 90 d8 ab 00 8c bc d3 0a f7 e4 58 05 b8 b3 45 06 }
{ d0 2c 1e 8f ca 3f 0f 02 c1 af bd 03 01 13 8a 6b }
{ 3a 91 11 41 4f 67 dc ea 97 f2 cf ce f0 b4 e6 73 }
{ 96 ac 74 22 e7 ad 35 85 e2 f9 37 e8 1c 75 df 6e }
{ 47 f1 1a 71 1d 29 c5 89 6f b7 62 0e aa 18 be 1b }
{ fc 56 3e 4b c6 d2 79 20 9a db c0 fe 78 cd 5a f4 }
{ 1f dd a8 33 88 07 c7 31 b1 12 10 59 27 80 ec 5f }
{ 60 51 7f a9 19 b5 4a 0d 2d e5 7a 9f 93 c9 9c ef }
{ a0 e0 3b 4d ae 2a f5 b0 c8 eb bb 3c 83 53 99 61 }
{ 17 2b 04 7e ba 77 d6 26 e1 69 14 63 55 21 0c 7d }

Comparando S-Box con la del fichero AES_tables.c

Las S-Boxes coinciden.

javi@javi-TP301UA:~/Documentos/Cripto/p2$
```

Como observamos en la salida, al compararlas con las tablas almacenadas en "AES_tables.c" ambas tablas calculadas son correctas. Además, si se comparan visualmente con las S-Boxes se confirma también que son correctas.

4.2.4 Uso del programa

Hemos seguido el formato especificado en el enunciado, que es el siguiente:

```
SBOX_AES {-C|-D} [-o file_out]
```

donde los parámetros son:

- el modo: -C o -D, para la S-Box directa o inversa respectivamente.
- el fichero de salida: -o seguido del nombre del fichero. Si no se introduce se usa la salida estándar.

De este modo, si se introduce en el terminal:

```
1 ./SBOX_AES -C
```

Obtendremos en el terminal la salida mostrada anteriormente para la tabla directa. Si por el contrario introducimos un fichero de salida:

```
1 ./SBOX_AES -C -o tabla_dir
```

La S-Box se imprimirá en el fichero proporcionado, pero el resultado de la comparación con la S-Box correspondiente del fichero "AES_tables.c" continuará mostrándose en stdout.

```
javi@javi-TP301UA:~/Documentos/Cripto/p2$ ./SB0X_AES -C -o tabla_dir
Tipo de tabla C/D: C Directa
Salida: tabla_dir
Comparando S-Box con la del fichero AES_tables.c

Las S-Boxes coinciden.

javi@javi-TP301UA:~/Documentos/Cripto/p2$ █
```